

jiram_catalog

A pedagogical review: two instruments, geometry, and scientific readiness

Implementation through 1e2723d — 7 September 2026

Companion to PEDAGOGICAL_REVIEW.md; every footer cites the review section it expands on.

Outline

What the tool is for

Three archive facts

Architecture

Deep dives

Figures from real data

Testing philosophy

How it was built

How to extend it

Glossary

Juno JIRAM, in one slide

- ▶ Two-channel infrared camera on Juno: L-band ($3.455\text{ }\mu\text{m}$, aurora, H3^+) and M-band ($4.780\text{ }\mu\text{m}$, thermal, day & night)
- ▶ Detector: $128\text{ lines} \times 432\text{ samples per band}$; 256-line LM products stack both bands, 10 gap lines removed
- ▶ Juno spins at 2 rpm (12 deg/s); JIRAM's despinning mirror holds one look steady per spin
- ▶ Initial JIRAM census: 85,108 archived camera frames (4 September build log); counts describe that snapshot

The archive's own labels do not tell you where a frame is pointed

They tell you which SPICE kernels were used to figure that out — computing it yourself is the first half of this project.

JunoCam adds a different measurement

- ▶ A spinning **pushframe camera**: an archived image contains successive 128×1648 framelets, with physical filter strips
- ▶ Native RED, GREEN, BLUE and METHANE are instrument channels; RGB is a viewing composite
- ▶ Its unsigned detector samples and per-frame timing require a separate adapter, not JIRAM's float reader or single-frame epoch
- ▶ The common workspace starts after each instrument has explicit geometry, validity, units and source identity

Current scope, 7 September 2026

Native JunoCam inspection is available. Failure exclusion and readiness checks decide which observations and calculations are accessible.

Repeat coverage determines the useful product

A small detector on a fast spacecraft gives two useful observing regimes. They describe *measured overlap and timing*, not a permanent division of Jupiter.

Repeated views of one patch

JIRAM polar campaigns motivate fixed-grid time stacks. Check the actual cadence and common valid area before treating them as motion data.

Individual mapped looks

Much of the available sampling motivates a strip library. A later visit can exist without a usable short-baseline sequence.

Regime 1: polar / repeat-view

- ▶ Repeated observations: verify unique exposures, positive cadence and common footprint; polar location alone is insufficient
- ▶ Natural product: a **region time stack** — every overlapping frame, reprojected onto one fixed grid, laid along a time axis
- ▶ Ground truth exists: Ingersoll et al. (2022, *Nature Astronomy*), 48 archived PJ4 frames → 4 published polar mosaics + wind vectors
- ▶ Built by: `region-stack`, `movie`, `export-goflow`

Regime 2: a library of mapped looks

- ▶ Useful even when a local sequence does not support motion retrieval
- ▶ Natural product: a **library** — looks on local tangent-plane grids, queryable by latitude, epoch and illumination
- ▶ Intensity structure can be compared across appropriately grouped observations; this does not require pretending they are a movie
- ▶ JIRAM paths: `strips`, `strip-stats`; JunoCam has its own native image-to-strip adapter

The two regimes, side by side

	repeated views	mapped-look library
product	region time stack	strip library
grid	fixed per region	fixed per strip
scientific question	change over a usable Δt	spatial intensity structure
evidence needed	cadence, navigation, shared mask	masks, units, resolution, grouping

Published PJ4 JIRAM maps and vectors validate a particular campaign. They do not provide a mission-wide or JunoCam wind-accuracy guarantee.

Three JIRAM facts, found the hard way

1. Archive labels lose all geometry after orbit 38
2. Image files are little-endian despite every label saying otherwise
3. Sequence numbering is 1-based, and frame 1 never arrives

Each one is enforced today by a specific line of code, not a comment. Get any of them wrong and geometry corrupts silently.

Fact 1: labels lose geometry after orbit 38

“label geometry is ABSENT for orbits ≥ 39 ... and for most 256-line dual-band products ... 64,139/85,108 frames have no label centre.”

—docs/build_log_2026-09-04.md, step 3d/3e

75% of that JIRAM census has no usable label geometry at all.

Fact 1: what it forced

- ▶ Label geometry could never be the foundation — only a spot-check
- ▶ `geometry.py + geo.py` → `frames_geo.parquet` became **mandatory** infrastructure, not a convenience
- ▶ Every downstream module reads `frames_with_geo()`; none fall back to label geometry, because for 3 in 4 frames there is nothing to fall back to

Fact 2: little-endian, despite every label

The JIRAM labels declare IEEE754MSBSingle / IEEE_REAL — big-endian, unambiguously.

Read big-endian:

spans -2.6×10^{38} to 3.1×10^{38} , 143 NaNs

Read little-endian:

spans -0.00017 to $0.5547 \text{ W m}^{-2} \text{ sr}^{-1} \mu\text{m}^{-1}$
— matches the published map's 0.0060–0.5545

Found independently twice, from two unrelated questions, both times by checking whether the numbers were physically plausible.

Fact 2: what it forced

- ▶ JIRAM RDR raw-pixel reads use '`<f4`' — the empirically verified type
- ▶ This rule is instrument-specific: JunoCam RDR uses `>u2`; EDR uses unsigned 8-bit counts (section 4.18)
- ▶ The original crawl-index spec still says “big-endian” — explicitly superseded, left as historical record

Rule of this repo: where code and an old spec disagree, the code wins, and the spec stays as a record of what was believed at the time.

Fact 3: frame 1 never arrives

“sequence_number is 1-based and frame 1 is never delivered ($seq_n = sequence_samples - 1$ always).”

—docs/build_log_2026-09-04.md, “Facts” after step 3d

Exact in the reconstructed JIRAM census: delivered sequences omit their first commanded frame. JunoCam has a different image/framelet identity model.

Fact 3: what it forced

sequence_number cannot be trusted across a real gap either.

index.assign_sequences instead detects boundaries from observed behaviour:

new sequence starts when:

gap since previous frame > 45 s # *one spin ~30.5 s*
or sequence_number fails to increase

Later JIRAM grouping (strip chunking, trackability pairing) trusts this reconstruction.
Do not apply its 45-second rule to JunoCam framelets.

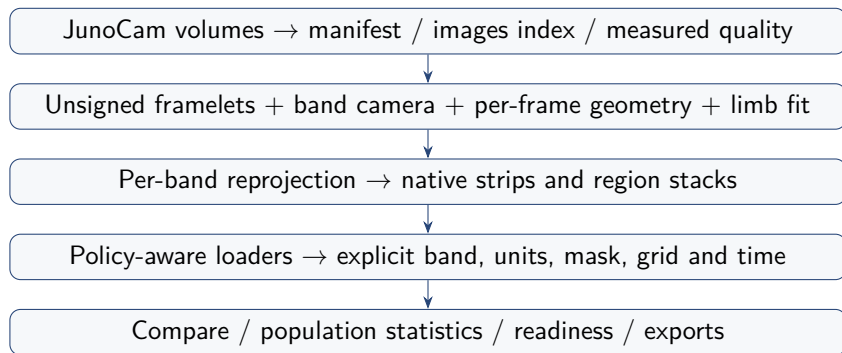
Layers, in build order

1. `pds.py` + `mirror.py` — crawl and download the archive
2. `labels.py` + `index.py` — parse labels, build `frames.parquet`
3. `geometry.py` + `kernels.py` — the SPICE engine
4. `geo.py` — geometry for every frame → `frames_geo.parquet`
5. `vicar.py` + `reproject.py` — ground-truth I/O, map grids, inverse camera
6. `regions.py` → `stacks.py` / `strips.py` — the two regimes
7. `movie.py`, `export_goflow.py`, `stats2d.py`, `trackability.py` — exports and analysis
8. `tracking.py` — the standing acceptance test, not a pipeline stage
9. `api/` + `frontend/` — browsing and orchestration; `science.py` supplies analysis preparation and `api/science.py` the research endpoints

One frame's path: archive to product

```
JIRAM PDS archive --(pds.py, mirror.py)--> mirrored .LBL/.IMG
  --(labels.py, index.py)--> frames.parquet (+ sequences)
  --(geometry.py, geo.py)--> frames_geo.parquet
  |
  +-- regime 1: select_frames -> reproject_frame -> region stack .nc
  | -> composite_sequences / movie.py / export_goflow.py
  +-- regime 2: unit_rows -> chunk_table -> build_strip -> strip .nc
  | -> stats2d.strip_statistics -> stats NetCDF
  |
  +-- GUI: browse + science API -> gui_cache/research/, exports/
```

A second adapter, shared scientific contracts



Where every product lives on disk

product	path (relative to mirror root)
label index	index/frames.parquet
SPICE geometry	index/frames_geo.parquet
SPICE kernels	spice/{lsk,pck,fk,ik,sclk,spk,ck}/
region stacks	regions/<region>/<band>_orbits<spec>_<level>.nc
goflow realizations	regions/<region>/goflow_<band>_orbits<spec>/rNNNNN/
strip NetCDFs & index	strips/orbitNN/<strip_id>.nc, strips/strips.parquet
strip statistics	strips/stats/ or gui_cache/stats_<id>.nc
GUI cache/exports	gui_cache/, gui_cache/exports/

JunoCam and research paths stay separate

All paths below are relative to the mirror root.

product family	location
archive manifest / every-version manifest	junocam/manifest/
image, geometry and quality indexes	junocam/index/
geometry and limb-fit cache	junocam/geometry_cache/
native region stacks	regions/<region>/junocam_*.nc
native strip products	strips/junocam/orbitNN/
bounded PDS reference audit	junocam/calibration_review/
new research caches and movies	gui_cache/research/
browser velocity-model exports	gui_cache/exports/

Source files are preserved. Export destinations include the selected sources and settings; a changed policy cannot reuse stale realizations as a new result.

Deep dives: what each one covers

For every module: purpose, key data structures, the algorithm, what a modifier must know, the validation gate, and the scars.

19 teaching sections; the original JIRAM deep dives retain their numbers:

- ▶ archive mirror & index (4.1–4.3)
- ▶ geometry & kernels (4.4–4.6)
- ▶ map machinery & regions (4.7–4.9)
- ▶ the two regimes (4.10–4.11, 4.13)
- ▶ validation & analysis (4.12, 4.14–4.15)
- ▶ the GUI, config (4.16–4.17)
- ▶ JunoCam adapter and shared scientific workflow (4.18–4.19)

Purpose: Apache listings → manifest → verified download.

- ▶ Only the Wget user agent is permitted under /PDS/; every download is a real wget subprocess
- ▶ Completeness = exact declared byte count from the sibling label, or last line END, or trailing </Product_Observational>
- ▶ -verify re-checks MD5 and re-downloads mismatches once

Scar: orbit50's listing alone is ~36 MB HTML — large enough to need retry-with-backoff, not a single timeout.

Purpose: one .LBL → one flat, typed dict — tolerantly.

- ▶ `pvl.load`, then a fallback grammar, then require the last line to be literally `END`
- ▶ A parse failure never raises: the row still exists, `parse_ok=False`, every geometry column `NaN`
- ▶ `"N/A"` (any case) is the *only* missing-value sentinel the archive uses
- ▶ `geom_band`: whichever band group has a numeric `CENTER_LATITUDE` — independent of which band the image data actually is

Purpose: multiprocessing label parse → frames.parquet, then segment sequences over the *whole* sorted table.

- ▶ orbit is float64 (4,697 nulls); **always use** orbit_dir, never orbit
- ▶ orbit disagrees with orbit_dir on 3,900 rows — all orbit03, labelled ORBIT_NUMBER=1; an archive property, not a bug
- ▶ sequence columns are recomputed over the combined table every run, never patched incrementally

Purpose: per pixel, per frame $\rightarrow$ lat/lon/range/emission/incidence/phase, no per-pixel Python loop.

```
x = ifov * (lines/2 + 0.5 - line) # 1-based line index
y = ifov * (samples/2 + 0.5 - sample) # 1-based sample index
d = (x, y, 1) / |(x, y, 1)| # unit ray, band frame
```

One vectorised numpy op over the whole (lines, samples, 3) grid — never per pixel. Then pxform: band frame $\rightarrow$ J2000 $\rightarrow$ IAU_JUPITER.

geometry.py: the aberration fix

`sincpt`'s `srfvec` and every pixel ray are *apparent* (light-time + stellar aberration). Both must be un-aberrated *the same way* before the ellipsoid intercept.

Original spec: un-aberrate rays only

per-pixel error vs. oracle: **4e-3 deg**

Corrected: un-aberrate `srfvec` too

per-pixel error vs. oracle: **8e-5 deg** — a 50× reduction

Order of the mistake: $|v|/c \approx 7 \text{ arcsec} \approx 4 \text{ km}$ on Jupiter's surface at perijove range.

- ▶ Juno spins at 12 deg/s: a 0.1 s epoch error moves the boresight > 1 deg
- ▶ For this JIRAM geometry model, use the label's START_TIME; the tested mid-exposure alternative was off by *tens of degrees*
- ▶ Engine is deliberately **mirror-blind**: models bus attitude only, not JIRAM's despinning mirror

Validated before writing code: mirror-blind SPICE matched 48 label centres to median 0.006 deg lat/lon, 0.017 deg emission, 28 km altitude.

kernels.py: SPICE kernel management

- ▶ Static set (leap-seconds, planetary constants, one frame/instrument kernel) + per-orbit CK/SPK named in the labels
- ▶ Operational server first, PDS SPICE archive as fallback (renames SPKs: `spk_rec_*` $\leftrightarrow$ `juno_rec_*`)
- ▶ `naif.jpl.nasa.gov` fails system-CA TLS; every fetch pinned to the certifi bundle

Open kernel coverage: the earlier six long CK gaps did not explain every failure. Orbits 38/55 lack attitude coverage; orbit 70 lacks trajectory coverage. The missing intervals remain unresolved.

geo.py: augmenting every frame

- ▶ spawn workers (never fork) — SPICE kernel pool is per-process global state
- ▶ One row per (frame, band half); a 256-line LM product → 2 rows
- ▶ A frame's computation failure never aborts the run: `geo_ok=False`, error text recorded, row still exists

`bore_lat` $\neq$ label `CENTER_LATITUDE` for a partial frame

`bore_lat`: the boresight's own intercept, NaN if the boresight misses.

Label centre: a *visible-area centroid* for a partial (limb) frame.

Comparing the two without filtering to fully-on-planet frames looks like a large geometric error that is really just two different definitions of “centre.” **Open:** L-band residual (median 0.024 deg) is roughly double M-band's (0.010 deg) — both inside the M-only gate tolerance; cause unresolved.

- ▶ JPL's older format: ASCII label *inside* the file, not a sidecar
- ▶ Label size (LBLSIZE) always read from the file, never hard-coded — the paper's own supplementary readers hard-code 12,800 bytes and silently misparse the 1,728-byte .tp4 vector tables
- ▶ EOL=1: a *second* trailing label after the data (TRACKER4's own processing history)

reproject.py: the inverse camera model

`project_to_pixels`: run `geometry.py`'s chain backwards.

```
(lat,lon) -> surface point -> IAU_JUPITER -> J2000  
  -> re-apply FORWARD stellar aberration (put it back!)  
  -> band frame -> invert pinhole -> fractional (line, sample)
```

Round-trip residual on a live frame: **1.22e-05 px**.

`reproject_frame`: walk the *output* grid, pull from the source — no output pixel unpainted, no input pixel double-scattered.

reproject.py: stereographic was wrong

candidate	centroid RMS (px)	sing. ratio	km/px
stereographic	0.828	1.00178	15.069
orthographic (ellipsoid)	0.339	1.00010	15.005

Every JIRAM paper checked calls the polar maps “stereographic.” The fit says otherwise, decisively: the map *is* the body-fixed equatorial plane, orthographically projected — no trig at all once you have the 3-D point.

Three candidate radii (polar, equatorial, $\sqrt{ac}$) were proposed for the fit.

They cannot be distinguished by residual at all

$\rho \propto R$ for every radial law — registration only ever measures the *ratio* $R/\text{km_per_px}$, never R alone.

Settled instead by requiring self-consistency with the one independently stated number: the label's own $\text{MPS}=15.0$.

regions.py: two grid rules

- ▶ `polar_ortho`: the published rule, generalised to either pole, any central meridian — `north_pole_paper` reproduces `PAPER_GRID` exactly
- ▶ `local_ortho`: tangent plane at an arbitrary centre, dropped onto the ellipsoid along the local normal — every mid-latitude strip's grid

A new named region (e.g. the Great Red Spot) is a **4-line YAML addition**, no code.

regions.py: the visibility subtlety

Naive test $(P - c) \cdot up > 0$ fails: the tangent plane touches the ellipsoid *only* at c , so this is ≤ 0 almost everywhere — near side and far side alike.

Correct test: use the surface normal at P itself

$visible = (\text{normal}(P) \cdot up) > -tol$

Exactly the silhouette condition a rendering engineer would recognise.

stacks.py: selection, three cost tiers

`select_frames`, cheapest test first:

1. Scalar quality cuts on `frames_geo.parquet` — free
2. Bounding-box prefilter against the region — free, approximate
3. Exact test: fresh geometry, every 4th pixel, must land on canvas — expensive, exact

Three tiers, three cost levels, cheapest first — run the expensive exact test only on what already survived the free filters.

- ▶ One geometry-only pass unions every frame's footprint box *before* any pixel is reprojected
- ▶ Output array allocated **once**, at the tight window's size — never at the full region canvas (a north_pole region is 3600×3600)
- ▶ spawn workers each return only their own frame's footprint box, not the shared window

Initial JIRAM PJ4 measurement: 294 frame steps, 2.25 GB. Sequence composite: 25 steps, more valid coverage than any one frame.

- ▶ **One whole-stack** brightness stretch, not per-frame — a per-frame stretch would *hide* the wobble that reveals a bad reprojection
- ▶ `export_goflow`: cuts variable-cadence stacks into constant-cadence *runs*; invalid pixels **zero-encoded** (not NaN) to match the downstream model's own convention
- ▶ No `u_mid/v_mid`: real data has no ground-truth velocity, unlike the synthetic sets this layout was designed to match

90% of a published map is exact-zero background; validity has to be a first-class concept, and the mask *depends on trial displacement*. 6 masked window sums $\times$ 1225 trial displacements per template — one at a time is hopeless.

Cost volumes

Each sum, at one fixed displacement, is a box filter over the whole tile — serves every template in that tile at once.

Measured throughput: **2,141 templates/s** on one core.

tracking.py: the reversed line axis

line axis	centre-on-footprint	full-template-on-footprint
direct	0.0%	0.0%
reversed	96.4%	91.4%

$\text{row} = \text{NL} - 1 - (\text{line} - 1)$ — settled unambiguously by footprint overlap.
0-vs-1-based index origin: genuinely underdetermined — the two conventions differ by 0.0005 px in the pooled result, “the tracking cannot separate them, which is the point.”

tracking.py: the gate numbers

	n	median (px)	frac < 1px
A: paper maps vs. TRACKER4	516,517	0.224	83.4%
B: our reprojections vs. TRACKER4	515,377	0.434	80.1%

The A→B gap (~ 0.2 px) is the honest cost of the full geometry+reprojection chain, on top of the tracker's own noise floor. **Scar:** 2 of 24 pairs (n0204aa/bb) sit near 1.04 px median — pooled statistic absorbs it, per-pair table reveals it.

strips.py: chunking a sequence

A single commanded sequence can slew tens of degrees while it runs. A chunk ends when:

- ▶ great-circle distance from the *chunk's first row* exceeds 12 deg, *or*
- ▶ the time gap exceeds 120 s

Chunks under `min_frames` are dropped but still consume a `chunk_index` — so a strip's name never shifts if the threshold changes later.

strips.py: resolution classes

RESOLUTION_CLASSES = (2, 3, 5, 7, 10, 15, 20, 30, 50, 70, 100, 150, 200, 300) km/px — log-nearest snap, ties to the finer class.

Why quantise at all

Two strips of the same class compare spectrum-to-spectrum, structure-function-to-structure-function, **with no resampling** — resampling would contaminate exactly the small-scale statistics being measured.

Initial JIRAM result: 289 strips, orbits 4+24, both bands, 392 MB. This is a historical build measurement, not the current mixed-instrument count.

Every diagnostic in this module follows one recipe, because a strip is a field with holes:

1. detrend over *valid* pixels only
2. fill invalid pixels with that same mean
3. taper the whole canvas (Tukey/Hann)
4. divide by $\text{valid_frac} \times \text{window_power}$

Result: a fully-valid, untapered field obeys Parseval exactly; a masked field is unbiased in *total* variance to first order.

Masking *convolves* the true spectrum with the mask's own spectrum — moves variance across k even when the total is right.

- ▶ Smooth, sparse holes: mild distortion. Measured: slope -2.97 unmasked vs. -2.92 with 15% masked
- ▶ Speckle masks: broadband leakage — **“no scalar correction can repair”** this (module's own words)

A documented, *measured* limitation, not hidden and not over-corrected.

$$b^2 = \frac{|\sum_s X_s(k_1)X_s(k_2)X_s^*(k_1+k_2)|^2}{\sum_s |X_s(k_1)X_s(k_2)|^2 \cdot \sum_s |X_s(k_1+k_2)|^2} \in [0, 1]$$

1.0 when the sum-mode phase is *locked* to the two parents' phases every segment; falls toward $1/n_{\text{segments}}$ when random. Test: same power spectrum, coupled vs.

independent third-term phase $\rightarrow b^2 > 0.5$ vs. $b^2 < 0.1$ at the same bin — detects coupling an ordinary spectrum cannot see at all.

trackability.py: before any pixel is reprojected

Pure numpy/pandas, no SPICE, no imagery. Candidate pair: same orbit/band, 90 s–6 h apart, boresights within a generous separation proxy.

The threshold alone over-admits

Dense polar mosaics: one boresight falls near *several* raster positions in the next sequence, not just its true repeat — verified numerically (closest wrong-position pair $<$ farthest correct-position pair).

Fix: keep only the **nearest partner per sequence** — on top of the threshold, not instead of it.

trackability.py: the best-baseline redefinition

Original: partner whose displacement is closest to 5 px

Revised: smallest dt among *trackable* partners

A sequence hours away could *coincidentally* land closer to 5 px than the true nearest repeat and win. “Smallest usable dt ” has no such failure mode. Initial JIRAM totals: 29,862 unit frames, 16,541 with a partner, 7,493 trackable at 30 m/s, 61 orbits. JunoCam trackability remains unassessed by this model.

Five views follow five scientific tasks

Explore	Find observations; inspect identity, footprint, date and band.
Time series	Inspect a fixed-grid stack; check cadence and export readiness.
Image library	Inspect strips; request individual or population statistics.
Compare	Align the view, blink sources and inspect registration diagnostics.
Coverage	Distinguish archive-known, local, navigated and eligible products.

One Zustand store and typed Arrow arrays connect the views. Legacy internal identifiers `catalog/poles/strips` remain for compatibility; the labels now describe tasks.

Leave room for the image, then reveal detail

- ▶ Explore starts with **coverage density**; swath outlines, coordinate conventions and selected footprints can be inspected explicitly
- ▶ A collapsible selection tray and metadata panels protect image area; controls wrap and statistics panels keep a usable minimum height
- ▶ Grayscale PNG stores value in R and validity in alpha; local colour-map lookup changes appearance without a new scientific calculation
- ▶ Orthographic navigation uses a scalar zoom, preserving aspect ratio; RGB uses a separate composite path

A selection should say what the next action means

- ▶ Stable product/half keys survive reordering; metadata shows version identity and quality reasons
- ▶ “Browse images from these orbits” transfers an *orbit filter*, not an exact claim that the listed strips contain every selected exposure
- ▶ Stack builds, movie renders and exports are jobs with visible progress; polling remains available during long work
- ▶ Coverage keeps excluded and unassessed JunoCam entries metadata-only, including direct thumbnail access

Interaction is part of scientific provenance

The currently named source and settings must describe the pixels actually on screen.

A late response must not relabel old pixels

Suppose RED finishes loading after the user has already selected GREEN.

1. Identify a request by source, frame, band/RGB, normalization, stretch, size and every other setting carried on the image request
2. Clear previously displayed pixels while a new uncached request is pending; otherwise the new label describes the old image
3. Accept completion only if its full request identity is still current

An out-of-order regression checks the race. A browser assertion must wait for actual visible pixels, not merely a changed control or an empty canvas.

Responsive browsing does not change the estimator

- ▶ Statistics start on request. A 6000×6000 native JunoCam strip can take several minutes; the interface says so while it is pending
- ▶ Preview metadata slices *before* loading array values; graticules and local-time contours use a bounded grid
- ▶ Display ranges share reads across normalization choices. Source signatures invalidate caches when inputs or policy change
- ▶ Native scientific normalization, FFTs and structure functions retain native pixels; no faster estimator was silently substituted

The NetCDF lock protects correctness, with a cost

- ▶ Concurrent NetCDF reads caused a real process crash. A shared lock now spans complete image, coverage and science routes and relevant jobs
- ▶ An image request can queue behind requested native statistics; serialization does not make every operation instantly responsive
- ▶ Health, job status, configuration and Arrow catalog reads remain outside that lock
- ▶ Single-flight table caches avoid duplicate cold loads; Arrow uses an explicit bounded pool instead of inheriting a pathological one-thread setting

Concurrency checks therefore verify both protected access and the independent status paths.

config.py + the subcommand pattern

- ▶ Precedence: explicit arg > env var > TOML file > built-in default — resolved *at call time*, never cached at import
- ▶ `add_subparser(subparsers) / run(args)`: a module owns its own CLI surface; `cli.py` wires it in afterward
- ▶ Enforces a project rule: executors never edit `cli.py`, `pyproject.toml`, `uv.lock` directly — exactly where concurrent milestones would collide

JunoCam: identify the observation before counting it

- ▶ `junocam/pds.py` discovers published volumes and builds an archive manifest; downloaded labels and images populate a separate local index
- ▶ A later `_V02` release can supersede `_V01`. They represent one observation, even if corrected label times differ by milliseconds
- ▶ Defaults select the **latest known version**, then require eligibility. They do not silently fall back to a clean older release
- ▶ An eligible older version can be inspected explicitly; derived products retain their stored source-version provenance

Archive discovery, local availability and scientific eligibility are three different counts.

JunoCam: an image is a sequence of band framelets

`junocam/images.py`: natural array shape

$(n_{\text{frames}}, n_{\text{bands}}, 128, 1648)$; bands cycle inside each frame in the label's `FILTER_NAME` order.

- ▶ RDR: unsigned big-endian 16-bit values (`>u2`), already linearized DN
- ▶ EDR: unsigned 8-bit companded counts; the SIS `SQROOT` lookup restores the linear count scale
- ▶ The table is not a fitted square-root formula: code 23 maps to 23, code 255 maps to 2879
- ▶ Shape and sample count are checked; summed products on a different detector grid are rejected

JunoCam: the camera is a band-dependent geometry

- ▶ `camera.py` reads focal length, distortion centre and radial distortion from the instrument kernel
- ▶ Array indices refer to pixel centres; the kernel's framelet coordinates start at a corner. The $+0.5$ conversion is deliberate
- ▶ Different filters occupy different focal-plane strips. One ground feature crosses them in different frames as Juno spins
- ▶ Reprojection can bring those observations together only with the correct camera *and* frame timing; a colour fringe is a diagnostic, not just a display defect

JunoCam: use an epoch for each frame

$$t_i = t_{\text{start}} + b_{\text{start}} + i(\Delta t_{\text{label}} + \delta t_{\text{IK}}) + \delta t_{\text{fitted}}.$$

- ▶ Commanded bands share a readout epoch within one frame; the next frame gets a new attitude and observer geometry
- ▶ `geometry.py` stores per-frame transformations for the inverse camera path
- ▶ `limb.py` fits one start-time offset per image against the predicted ellipsoid limb; it cannot absorb arbitrary frame-to-frame jitter or a wrong timing rate

At Juno's spin rate, millisecond timing errors can produce visible subpixel shifts.

JunoCam: a limb fit is evidence with limits

1. Interpolate the predicted ray/ellipsoid discriminant through zero: a smooth boundary avoids whole-row quantization
2. Detect the observed edge, fit an offset, then detect and fit again; compare before/after residuals on the same final point set
3. Inspect entering and leaving limbs separately. Haze above the reference ellipsoid can shift an apparent limb without a pointing error

The PJ4 assessment records limb-height and interframe-rate residuals. It retains the kernel timing rate; a low fitted residual does not establish independent navigation accuracy.

JunoCam: reproject each band and preserve its support

- ▶ `reproject.py` walks output cells through each frame's stored inverse transform, then bilinearly samples the appropriate filter strip
- ▶ Facing-surface, detector bounds and photoactive-column tests restrict valid contributions
- ▶ Per-band sums and hit counts produce a mean where observed and NaN where there is no support
- ▶ `stacks.py` and `strips.py` write fixed-grid products; masks, bands and source identities travel with the pixels

A displayed RGB composite is assembled from those physical bands; it is not a fifth measured channel.

JunoCam: quality is an access decision with evidence

Excluded	Documented affected images/intervals, measured signal failures or failed geometry; pixels withheld.
Unassessed	Incomplete metrics or no affirmative unaffected-image evidence; pixels withheld.
Eligible	Complete clean metrics and supported evidence under the versioned policy; navigation status remains explicit.

failure-exclusion-v1 keeps instrument, signal and navigation reasons distinct. A legacy A/B/C grade or “post-anneal” epoch does not establish recovery.

Loaders enforce this for images and derived analysis/export access. No browser override reveals withheld pixels.

JunoCam: what the current eligible sample supports

- ▶ At 1e2723d: **72 preferred eligible PJ4 observations** in the current mirror
- ▶ Compared with the earlier 93 mapped observations, 21 methane observations are additionally withheld by existing bloom flags; no RGB observation is removed by this additional cut
- ▶ The bloom metric is conservative signal evidence, not proof of a documented hardware event in every flagged image
- ▶ The polar stack contains **three unique observations**; gaps are about 577 and 243 seconds, outside a 5% constant-cadence tolerance

Viewing those three maps is useful. Calling them an export-ready regular three-frame sequence would be incorrect.

PDS “calibrated” is a derived reference here

- ▶ The ML collection preprocesses, flattens, reprojects and mosaics native inputs before generating Hubble-equivalent channels
- ▶ Its five channels include predicted UV and methane values; these are not five simultaneous native measurements
- ▶ Useful role: broad morphology and method comparison. Native motion, intensity spectra and cross-instrument radiometry require separate validation
- ▶ A plausible generated image does not establish repair of an instrument failure or make its damaged input eligible

Coverage links to the collection as a reference. No automatic native import or repaired-data certification is enabled.

PDS reference: what the bounded audit actually measured

- ▶ Inventory: 37,386 tile identifiers across PJ13–36; overlapping tiles are not that many independent exposures
- ▶ Three sampled PJ13 TIFFs: $256 \times 256 \times 5$, float32, 62.5 km grid spacing, LAEA with planetographic latitude
- ▶ All three labels have nil observation times. The sampled mosaic spans about 130 minutes; no per-pixel timing was established
- ▶ Actual FITS rows already contain values below 1; a guide's extra division by 255 would mis-scale them. Zeros and the labelled invalid sentinel require mask reconciliation

`docs/reports/junocam_calibrated_assessment_2026-09-07.md` records sources, hashes and limits. Three tiles plus headers/rows cannot certify the entire collection.

Choose the physical quantity before its colour scale

- ▶ `science.py` selects one physical band, validates the grid, prepares native arrays and records provenance
- ▶ JIRAM measures spectral radiance; native JunoCam products here carry DN. An illumination transform does not turn DN into calibrated I/F
- ▶ RGB is accepted for viewing. Statistics, motion-model exports and physical-band movies require an actual available channel
- ▶ Brightness stretch maps values to display levels; scientific normalization changes the field whose statistics are calculated

A plot labelled by instrument, band, units and normalization names a reproducible quantity.

Illumination correction has a domain of validity

For reflected light, write $\mu_0 = \cos i$ and $\mu = \cos e$.

$$I_{\text{Lambert}} = I/\mu_0, \quad I_{\text{Minnaert}} = I/(\mu_0^k \mu^{k-1}).$$

- ▶ Incidence is required; Minnaert also requires emission. JunoCam masks incidence $\geq 88^\circ$; cosine floors guard division near the limb/terminator
- ▶ These models reduce illumination trends; they do not establish absolute radiometry or preserve every spatial statistic
- ▶ JIRAM thermal radiance retains nightside emission. Reflected-light Lambert/Minnaert requests are rejected for it

The same smoothing number can mean a different scale

`flat:sigma` divides by a Gaussian low-pass background and normalizes the result. Sigma is expressed in *the array's pixels*.

Image preview

Operates after serving stride; sigma is in preview pixels. It is a viewing aid.

Analysis and export

Operates on native pixels before scientific calculations; the recipe records this choice.

If a preview keeps every fourth pixel, sigma 10 spans four times the physical distance of native sigma 10. Compare normalization groups using their recorded physical context.

Readiness checks a sequence, not a wind result

1. Deduplicate source versions; select a physical band and validate pixel scale and any supplied regular, square grid coordinates
2. Require unique observations with strictly positive time gaps
3. Find runs of at least three frames whose gaps satisfy the cadence tolerance
4. Apply the selected normalization and require pixels valid in every frame of a run

Report observation count, gaps, cadence runs and explicit failure reasons. A cadence run can contain several sliding triples; its count is not the number of all triples.

An export carries the recipe that made its arrays

- ▶ Record source versions, timestamps, physical band, units, grid, masks, normalization, policy and software revision, plus available kernel provenance
- ▶ Native preparation precedes the existing velocity-model layout; invalid samples remain zero-encoded according to that layout
- ▶ Source files are preserved; output identities include filtered sources and settings, and an explicitly named destination must be empty
- ▶ A ready export establishes layout and cadence compatibility. It does not validate navigation, learned-model behavior or resulting wind accuracy

Compare: shared coordinates before a numerical shift

- ▶ Linked pan/zoom, split/blink, shared stretch and common-mask overlay make discrepancies visible
- ▶ Registration requires equivalent projection and physical coordinates. Incompatible grids return a reason, not an implicit reprojection
- ▶ Masked NCC searches bounded displacements on an at-most-512-pixel sampled grid; means and variances use the shifted mask intersection
- ▶ Normalization still reads native arrays *before* sampling. A bounded FFT does not guarantee bounded native I/O

Insufficient support, weak variance or a peak at the search boundary is reported explicitly.

Compare: what the displacement and uncertainty mean

Positive $(\Delta y, \Delta x)$ describes pattern displacement from the *left image to the right image*, in native map pixels. Sampling stride and sampled km/pixel accompany it.

$$d_{\text{expected}} = \frac{U|\Delta t|}{p}, \quad \sigma_U = \frac{\sqrt{2} \sigma_{\text{nav,px}} p}{|\Delta t|}$$

Here p is native metres/pixel. The uncertainty expression assumes independent, user-supplied per-image navigation errors; absent input stays unknown. Registration measures a pattern shift, not a validated wind field. Cross-band morphology and scene evolution can displace the correlation peak.

Population spectra: count independent passes

1. Group by instrument, physical band, units, native resolution class and normalization
2. Count shared source observations once; conservatively exclude a strip that partly duplicates an already represented source set
3. Average strips within each pass, then give each independent pass equal weight
4. Estimate standard error from those pass means; with only one pass it is unknown, not zero

This prevents overlapping strips or processing versions from manufacturing a larger effective sample size.

A slope fit should travel with its mask and fit range

- ▶ Inspect valid fraction, connected components, boundary fraction, dynamic range and row discontinuity before interpreting a spectrum
- ▶ Fit only positive finite bins over an explicit k range; the default stays inside the Nyquist disc
- ▶ Pixel spacing and Nyquist wavelength are known; effective resolution remains unassessed without a transfer-function measurement
- ▶ Export numeric values and a JSON recipe alongside SVG/PNG figures; compare normalization sensitivity as distinct groups

These are **intensity-variance spectra**, not kinetic-energy spectra. Scalar mask compensation cannot undo spectral leakage.

Matches and vectors require different kinds of evidence

- ▶ Cross-instrument matches use time proximity and seam-aware spherical footprint boxes: approximate candidates, not exact pixel overlap or common cloud height
- ▶ A vector overlay requires explicit source stack, time, projection, `map_xy` basis, kilometre coordinates and m/s units in an existing product
- ▶ The current mirror lacks a generic vector product with that exact association; the normal result is unavailable with a reason
- ▶ Published PJ4 TP4 vectors remain in the established validation path. They are not silently attached to an arbitrary displayed stack

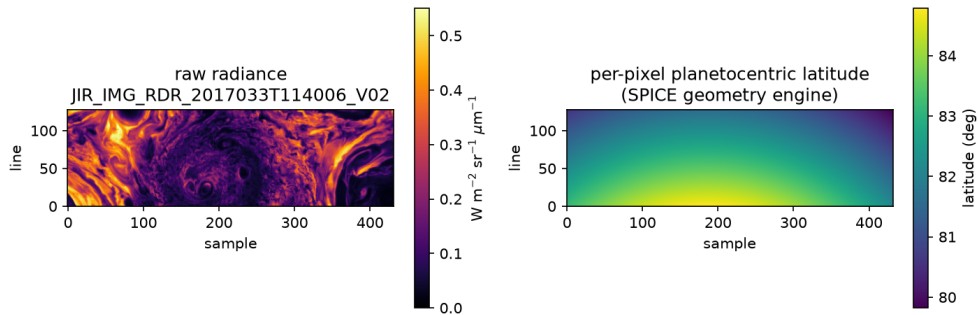
Six figures, all from real data on the mirror

Generated by `docs/pedagogy/make_figures.py`, reusing `jiram_catalog` directly
— no module modified.

- ▶ (a) one raw frame beside its computed latitude map
- ▶ (b) the published n01a map vs. our reprojection, with the difference
- ▶ (c) one mid-latitude strip with its graticule
- ▶ (d) that strip's isotropic spectrum and structure function
- ▶ (e) the trackability heatmap
- ▶ (f) a sketch of the SPICE frame chain and the pinhole model

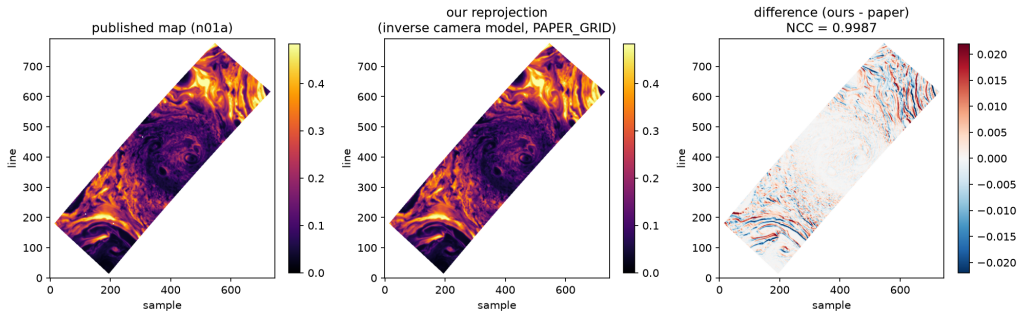
(a) A raw frame and its computed geometry

Figure (a): one JIRAM camera frame and its computed geometry



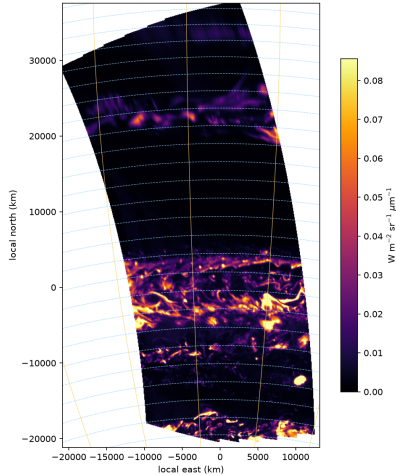
(b) Reproducing a published map from raw bytes

Figure (b): reproducing one published perijove-4 map from the raw archive frame



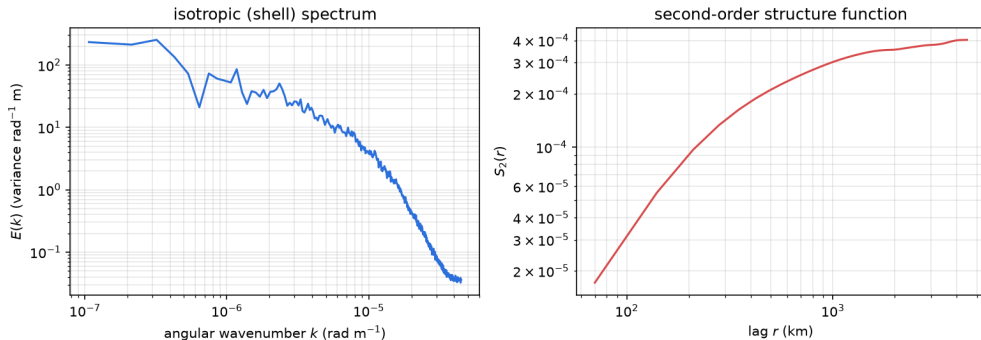
(c) A mid-latitude strip, with its graticule

Figure (c): strip 24_M_2019360T202519_01
orbit 24, band M, 70 km/px, centre -35.7 N, 343.6 E
(blue: latitude every 2 deg; gold: longitude every 10 deg)

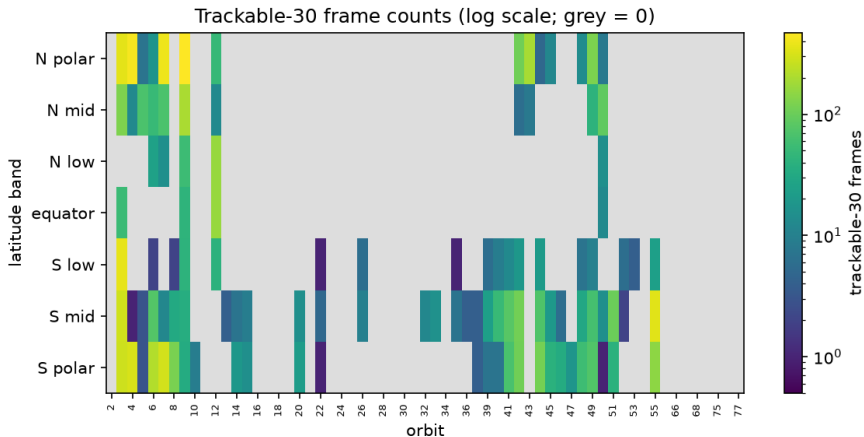


(d) That strip's spectrum and structure function

Figure (d): masked statistics of strip 24_M_2019360T202519_01

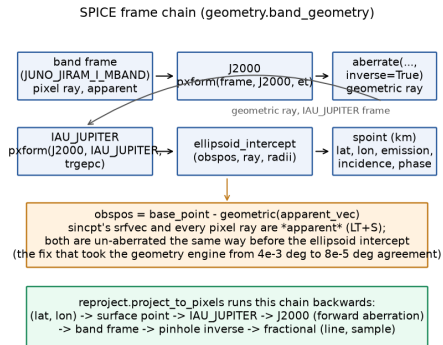


(e) The trackability heatmap



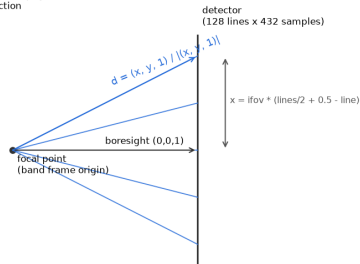
(f) The SPICE frame chain and the pinhole model

Figure (f): the SPICE engine's forward chain, and its inverse in reproject.py



pixel_directions: pinhole model in the band frame

y (samples) points out of the page;
x (lines) is the spin direction



Offline tests vs. gates

Offline tests

`test*_offline.py`

no mirror, no network

`JIRAM_SKIP_GATES=1`

synthetic fields, known answers

Gates

`test_gate_*.py`

physical gates: real mirror / paper data

contract gates: known synthetic answers

browser gates: complete live interactions

Gates are read-only to every executor, always. Passing a gate by editing it is task failure — stated in three separate project documents.

Where acceptance evidence comes from

1. The 48-frame PJ4 fixture — numbers a peer-reviewed paper actually published
2. The published maps and TRACKER4 tables themselves, read directly with `vicar.py`
3. A *fresh*, independent SPICE oracle call — the only kind that can catch a bug in the *vectorisation itself*
4. Known synthetic shifts, masks, spectra and cadence — explicit answers for the new workflow contracts; these complement physical validation

Wrong gate #1: the geo consistency report

All-orbit residuals folded together well-conditioned frames and *partial* (limb) frames, where the label's "centre" is a visible-area centroid, not a boresight intercept.

"geo gate rewritten by lead (well-conditioned median 0.0078 deg; all-orbit report median was inflated by partial/limb frames)"

—build log, step 8a'

Fix: restrict to fully-on-planet, low-emission frames before computing the statistic.

Wrong gate #2: the best-baseline definition

Original `best_dt_s` rule (“closest to 5 px displacement”): 14/15 gate checks passed on first run — one failure, from PJ4’s dense polar campaign letting a far-in-time sequence coincidentally win. **Response: investigate, don’t loosen the gate.**

Redefined to “smallest dt among trackable partners” — no equivalent failure mode; gate tolerance widened to reflect real cadence spread (90 s), not to hide the bug.

The general rule

When a gate fails

Investigate why. Never relax the threshold or edit the gate to match whatever the code produced.

“If a threshold is not met, do not tune parameters to pass; investigate ... and report what limits agreement.”

—docs/specs/2026-09-05_tracking.md

Both wrong-gate stories: the *code* was never the bug — the gate’s own definition was measuring the wrong thing.

Validation snapshot: know what each green check proves

At 1e2723d, recorded on 7 September 2026:

- ▶ 339 offline Python tests passed (49 skipped); 135 frontend unit tests passed
- ▶ Eight focused live API/policy/science gate checks passed; the full production-bundle browser gate passed
- ▶ New known answers cover shifts, masks, power laws, cadence, grouping, uncertainty, policy enforcement and out-of-order image responses
- ▶ The preserved older JunoCam gate has two passes and one count conflict: it expects at least 100 products; current policy admits 72 unique preferred observations

The count conflict is reported, not hidden by duplicate versions or weaker exclusions. These checks do not establish archive-wide radiometric or wind accuracy.

The loop, per milestone

1. **Facts first** — reconnaissance reports numbers, never designs
2. **Spec** — goal, files in scope, every decision explicit, exact validation block
3. **Gate before dispatch** — written by the lead, against ground truth
4. **Dispatch** — one executor, fresh context, the spec is the whole brief
5. **Verify** — scope check, gates run by the lead (not trusted from the report)
6. **Record** — outcome, executor, wall time, gate result → build log

Every spec ends: stop and list ambiguities rather than guess; list every judgment call.

What each executor tier actually did

- ▶ **Reconnaissance & prose:** lighter tier — 230k tokens settled the two facts the whole design rests on
- ▶ **Mechanical code, airtight spec:** cheapest executor that has passed a gate here — crawler + index, one pass
- ▶ **Bounded numerical/geometric work, SPICE/projection oracle, fresh-context review:** stronger tier — every milestone after the geometry engine, 15–45 min each, 1–3 genuine ambiguities each, resolved sensibly

The stalls, and the ambiguity stops that worked

The geometry engine, dispatched three times to the cheaper tier:

- ▶ Attempt 1 & 2: hit a genuine spec ambiguity, **stopped correctly** — exactly right
- ▶ Attempt 3: stalled **2h12m**, produced nothing — the failure mode with no automatic detector

Re-routed to the stronger tier: working engine, aberration fix included, in **16 minutes**.

Lesson: stopping on ambiguity $\neq$ failure. Silent non-progress needs a wall-clock budget, watched by whoever dispatched the work.

Errors executors caught, twice

- ▶ **Byte order** (little-endian, section 2.2): found *inside* the geometry-augmentation milestone by an executor trusting physical plausibility over the labels; the reprojection gate, written with the documented byte order, failed at the same moment and confirmed it independently
- ▶ **Stellar aberration** (section 4.4): a specific, measured $4\text{e-}3$ deg $\rightarrow$ $8\text{e-}5$ deg fix, found by an executor checking its own output against the same oracle the gate would later check

Neither was caught by a human re-reading the spec more carefully — both were caught by checking output against reality.

What this teaches about delegating numerical work

1. An executor that **stops on genuine ambiguity** is working correctly — never penalise this
2. A gate is only as good as its own definition of ground truth — distrust a failing gate's assumptions, not just the code under it
3. Genuine-oracle numerical work (a published number, an independent SPICE call, a known synthetic answer) is exactly where dispatch-then-verify-hard catches real errors — *in both directions*

The browser revision exposed scientific bugs too

- ▶ An apparently harmless version suffix doubled observations and could fabricate cadence; identity belongs in source loaders and exports
- ▶ A band selector could advance before the pixels arrived; the fix required full request identity and immediate clearing, not a new label
- ▶ Concurrent NetCDF operations crashed the process; route/job serialization restored correctness while leaving a visible queueing cost
- ▶ A slow contour path loaded a full large field; bounded display reads improved usability without changing native statistics

The useful lesson is to test a scientific interaction through its complete source-to-pixel or source-to-export path.

Worked example: two extensions, chosen to contrast

1. A new named region: `great_red_spot` (a `local_ortho` patch)
2. A new statistic: skewness of the radiance field, $\langle (f - \mu)^3 \rangle / \sigma^3$

Same loop as every real milestone: facts $\rightarrow$ spec $\rightarrow$ gate $\rightarrow$ module $\rightarrow$ wiring $\rightarrow$ docs.

Step 1–2: facts, then the spec

Facts first: the storm's approximate centre lat/lon at a reference epoch; which orbits actually cover it. For skewness: run it once by hand on a real strip, check the existing `_prepare` masking helper is sufficient. **Spec:** files in scope, a read-only list, the exact API (`skewness(field, valid=None)` -> float, matching `structure_function`'s own calling convention), the validation block.

Step 3: the gate, written first

Region: no published storm ground truth exists — gate is *structural*: registry loads, round-trips to 1e-6 px, real-mirror selection is non-empty. Weaker than a scientific gate, and the spec should say so.

Statistic: a *real* numerical gate — synthetic field with known skewness, tight tolerance, plus a symmetric-field negative control at zero.

Step 4–6: module, wiring, docs

- ▶ Region: **4-line YAML addition** to `configs/regions.yaml` — no Python at all
- ▶ Statistic: `skewness` added next to `structure_function`, reusing `_prepare`; one new scalar attr on `strip_statistics`'s output
- ▶ Wiring: region needs none (`RegionGrid.from_yaml` is a generic lookup); statistic needs none beyond its own module (already owns its subcommand)
- ▶ Docs: one line each in `data_products.md`, `architecture.md`, `open_items.md` (no ground truth yet)

What the asymmetry teaches

- ▶ Region: almost no code, because `regions.py` was designed from the start to generalise past its initial four entries
- ▶ Statistic: real code, but slotted directly into an existing, well-established pattern — no new architecture needed

Neither extension needed a new pattern invented on the spot — that is itself a finding about the codebase, not just a tutorial.

Glossary: SPICE

- ▶ **SPICE**: NAIF's toolkit for spacecraft geometry from loaded *kernels*
- ▶ **Kernel** (SPK/CK/FK/IK/SCLK/LSK/PCK): trajectory / attitude / frames / instrument / clock / leap-seconds / planetary-constants files
- ▶ **Furnsh** / **kernel pool**: SPICE state is global per process — why every parallel worker here uses `spawn`, never `fork`
- ▶ **Ephemeris time (ET)**: SPICE time scale. JIRAM uses label start time; JunoCam adds instrument timing terms per frame and any fitted offset
- ▶ **Sincpt** / **pxform**: surface-intercept and frame-rotation calls; `geometry.py` vectorises the same computation over a whole pixel grid
- ▶ **Light time** / **stellar aberration ("LT+S")**: finite light speed, and the observer's own velocity bending apparent direction — getting the *inverse* applied consistently is section 4.4's central subtlety

Glossary: geometry conventions

- ▶ **Planetocentric** latitude: angle from the equatorial plane to the vector from the body's *centre* — the convention of native repository grids
- ▶ **Planetographic** latitude: uses the local surface normal instead — different on an oblate body; the external ML-calibrated reference requires conversion
- ▶ **East-positive longitude, [0,360)**: this project's fixed convention, matching the archive's own labels
- ▶ **System III**: Jupiter's body-fixed rotating longitude system; IAU_JUPITER *is* System III for this project's purposes
- ▶ **Emission / incidence / phase angle**: angle to observer / to Sun / between the two, at the surface point
- ▶ **Orthographic / stereographic / equidistant / Lambert**: four radial laws relating map distance to true angular distance; the published maps use orthographic, despite the literature calling them stereographic

Glossary: archive & this codebase

- ▶ **PDS3 / PDS4**: the archive's label syntax (PDS3 text) and bundle layout (PDS4 directories) — not the same axis
- ▶ **RDR / EDR**: reduced / experimental data record. Processing level alone does not establish units: JIRAM RDR is radiance; native JunoCam remains DN
- ▶ **VICAR**: JPL's older self-contained image format — used here only because the ground-truth maps and vector tables ship in it
- ▶ **NCC**: normalised cross-correlation, 1.0 for identical patches — the tracker's match score *and* the general “does our reprojection match the paper” figure of merit
- ▶ **geo_ok / on_planet_frac / bore_***: SPICE succeeded at all / fraction of pixels on the ellipsoid / evaluated at the boresight specifically — consistent naming across every table in this codebase

Glossary: the new scientific contracts

- ▶ **Framelet**: one filter strip's 128-line readout; many framelets make one JunoCam archived image
- ▶ **Observation identity**: the underlying exposure identity, independent of processing-version suffix
- ▶ **Eligibility**: evidence-based access under the current policy; distinct from absolute radiometric calibration
- ▶ **Readiness**: format, grid, cadence and mask compatibility for an operation; distinct from physical accuracy
- ▶ **Normalization / stretch**: transformation of the analysed field / mapping to display brightness
- ▶ **Independent pass**: the population uncertainty unit; overlapping strips are not independent replicates

Where to read next

full detail	<code>PEDAGOGICAL_REVIEW.md</code>
module-by-module facts	<code>docs/architecture.md</code> , <code>docs/decisions.md</code>
schemas	<code>docs/data_products.md</code>
command reference	<code>docs/usage.md</code>
current browser/science use	<code>docs/research_workflow.md</code>
open questions	<code>docs/open_items.md</code>

This deck and its figures: `docs/pedagogy/`